

דוח סיום הפרויקט

מגישות:

כרמל חאוזי 313557480

דני לין 322893736

סקירה של raytrace

ה-Raytrace benchmark מממש אלגוריתם שמטרתו ליצור תמונה דו-ממדית של סצנה תלת-ממדית. הסצנה מכילה מצלמה, עצמים ומקורות אור, והאלגוריתם מחשב עבור כל פיקסל בתמונה הפלט מה ניתן לראות מהמצלמה בכיוון המתאים לאותו פיקסל. התהליך מתבצע על ידי שליחת קרניים מהמצלמה אל תוך הסצנה. עבור כל פיקסל בתמונה נוצרת קרן שמתחילה במיקום המצלמה ועוברת בכיוון המתאים לאותו פיקסל. פונקציית render עוברת על כל הפיקסלים בתמונה, מחשבת את כיוון הקרן המתאים לכל פיקסל, יוצרת אובייקט Ray, שולחת אותו לפונקציה rayColour, ולבסוף שומרת את הצבע שהתקבל ב-Canvas.

הסצנה ב-benchmark כוללת שבע ספרות, משטח מישורי מסוג Halfspace ושני מקורות אור. עבור כל קרן, הפונקציה rayColour בודקת את החיתוך שלה עם כל העצמים בסצנה באמצעות הפונקציה intersectionTime של כל עצם. כל בדיקה מחזירה את הפרמטר t , שמייצג את מיקום נקודת החיתוך לאורך הקרן, או מציינת שאין חיתוך. לאחר חישוב החיתוכים, הפונקציה firstIntersection בוחרת את החיתוך התקף הקרוב ביותר לנקודת ההתחלה של הקרן, כלומר את החיתוך בעל ערך t הקטן ביותר מבין החיתוכים התקפים. אם הקרן אינה פוגעת באף עצם, הפיקסל מקבל את צבע הרקע. אם נמצא חיתוך, מחושבת נקודת הפגיעה על העצם ולאחר מכן מחושב צבע הפיקסל בהתאם לעצם ולתאורה בסצנה. בנוסף, נבדק האם הדרך בין נקודת הפגיעה למקור האור חסומה על ידי עצם אחר, וכך ניתן לקבוע האם הנקודה נמצאת בצל.

הקוד תומך גם בהשתקפויות. כאשר קרן פוגעת במשטח מחזיר, נוצרת קרן חדשה בכיוון ההחזרה, והפונקציה rayColour נקראת שוב עבור הקרן החדשה. לכן תהליך חישוב הצבע הוא רקורסיבי. עומק הרקורסיה מוגבל כדי למנוע יצירה בלתי מוגבלת של קרניים מוחזרות. בסופו של התהליך, הצבע שחושב עבור כל קרן משמש כצבע של הפיקסל המתאים בתמונה הדו-ממדית.

הקוד משתמש בשלוש ספריות Python עיקריות. הספרייה math משמשת לביצוע חישובים מתמטיים הנדרשים באלגוריתם, כגון חישוב שורש ריבועי ופונקציות טריגונומטריות. הספרייה pyperf משמשת להרצת ה-benchmark ולמדידת זמן הביצוע שלו. הספרייה array משמשת לשמירת תמונת הפלט: לאחר חישוב הצבע של כל פיקסל, ערכי ה-RGB שלו נשמרים במערך השייך ל-Canvas.

הקוד משתמש במספר מבני נתונים עיקריים. רשימות (list) משמשות לשמירת העצמים ומקורות האור בסצנה, כאשר כל עצם והמשטח שלו נשמרים כ-tuple. בנוסף, המחלקה Canvas מייצגת את תמונת הפלט ושומרת את ערכי ה-RGB של הפיקסלים באמצעות מערך מהספרייה array.

ניתוח ראשוני

כדי להבין אילו חלקים ב-benchmark צורכים את רוב זמן הריצה, הרצנו את הגרסה המקורית עם profiler ויצרנו Flame Graph. ה-Flame Graph מציג את שרשרת הקריאות בין הפונקציות, כאשר ככל שהאזור של פונקציה רחב יותר, כך חלק גדול יותר מזמן הריצה עבר דרך אותה פונקציה והפונקציות שהיא קוראת להן.

בבדיקה שביצענו, 30 הרצות של render לקחו בסך הכול כ-56.66 שניות. מתוך ה-Flame Graph ניתן לראות שחלק גדול מאוד מהזמן עובר דרך rayColour. הפונקציה נקראה לפחות 300,000 פעמים ישירות מתוך תהליך ה-render, והזמן שלה יחד עם הפונקציות שנקראות מתוכה היה כ-51.32 שניות, כלומר כ-90.6% מזמן ה-render. זה הגיוני מכיוון ש-rayColour היא הפונקציה המרכזית שמטפלת בקרן: היא בודקת חיתוכים עם העצמים, מוצאת את החיתוך הקרוב, מחשבת את צבע המשטח, ובמקרה של השתקפות יכולה גם להפעיל את עצמה שוב עבור קרן חדשה.

צוואר בקבוק נוסף שנראה בבירור הוא חישוב החיתוך עם ספרות. הפונקציה Sphere.intersectionTime נקראה כ-5.38 מיליון פעמים במהלך הבדיקה. הזמן שהפונקציה עצמה ביצעה בו חישובים היה כ-6.84 שניות, וכאשר כוללים גם את הזמן של הפונקציות שהיא קוראת להן, מתקבל זמן של כ-22.87 שניות. ביחס לזמן הכולל של render מדובר בכ-40.4%. כלומר, חלק משמעותי מזמן הריצה קשור לבדיקות חיתוך בין קרניים לספרות.

ב-Flame Graph ניתן לראות מסלול משמעותי דרך visibleLights, שהזמן שלו יחד עם הפונקציות שנקראות ממנו הגיע לכ-19.27 שניות, כלומר בערך 34% מזמן הריצה שנמדד. מתוך מסלול זה, רוב הזמן עובר דרך _lightsVisible, עם כ-19.0 שניות. הפונקציה בודקת עבור מקור אור האם קיים עצם בין נקודת הפגיעה לבין האור, ולכן היא מבצעת שוב בדיקות חיתוך עם העצמים בסצנה.

בהשוואה לחישובי החיתוך עם ספרות, הפונקציה Halfspace.intersectionTime נקראה 765,030 פעמים בלבד, והזמן שלה יחד עם הפונקציות שנקראות מתוכה היה כ-1.07 שניות. לכן התרומה שלה לזמן הריצה הכולל קטנה משמעותית מזו של Sphere.intersectionTime, ובהמשך הניתוח לא נתמקד בה.

חשוב לשים לב שהאחוזים האלה לא מתחברים אחד לשני, מכיוון שהפונקציות קוראות אחת לשנייה. לדוגמה, rayColour יכולה להגיע ל-visibleLights, ו-visibleLights קוראת ל-lightsVisible, שבתורה קוראת ל-lightsVisible.intersectionTime. לכן אותו זמן ריצה יכול להופיע כחלק מהזמן של כמה פונקציות שונות.

בסך הכול, ה-profilng וה-Flame Graph הראו שרוב זמן הריצה מרוכז במסלול של חישוב צבע הקרן, ובתוכו בעיקר בבדיקות חיתוך עם עצמים ובחישובי התאורה. בנוסף, מספר

הקריאות הגבוה מאוד לפונקציות אלה מראה שגם פעולות יחסית קטנות שחוזרות מיליוני פעמים יכולות להשפיע משמעותית על זמן הריצה הכולל.

אופטימיזציות

בגרסה שעברה אופטימיזציה בוצעו שני שינויים עיקריים, שנועדו לשפר את הביצועים בלי לשנות את האלגוריתם המרכזי של ה-Raytrace.

האופטימיזציה הראשונה הייתה שיפור של מבנה הנתונים של האובייקטים באמצעות הוספת `__slots__` למחלקות מרכזיות כמו `Point`, `Vector`, `Sphere`, `Halfspace` ו-`Ray`. ב-Python אובייקט רגיל שומר בדרך כלל את השדות שלו באמצעות `dict`. השימוש ב-`__slots__` קובע מראש אילו שדות קיימים בכל מחלקה, ולכן אין צורך ליצור ולנהל מילון תכונות נפרד לכל אובייקט. כתוצאה מכך נחסך זיכרון, וגם הגישה לשדות יכולה להיות יעילה יותר מכיוון שאין צורך לבצע חיפוש במילון. מכיוון שב-Raytrace נוצרים ונגישים מספר רב של אובייקטים מהמחלקות האלה, השינוי יכול להפחית גם את צריכת הזיכרון וגם את זמן הריצה.

האופטימיזציה השנייה הייתה שיפור לוגי בפונקציה `lightsVisible`. בקוד המקורי נוצר אובייקט `Ray` חדש בתוך הלולאה שעוברת על כל העצמים בסצנה. במהלך כל האיטרציות של אותה לולאה, הנקודה `p` ומקור האור `l` אינם משתנים, ולכן למעשה נוצרת שוב אותה קרן. בנוסף, יצירת `Ray` כוללת גם נרמול של וקטור הכיוון, ולכן בכל איטרציה בוצעו שוב גם יצירת אובייקט וגם חישוב שכבר בוצע קודם.

בגרסה המשופרת הקרן נוצרת פעם אחת לפני הלולאה ולאחר מכן אותה קרן משמשת לכל בדיקות החיתוך מול העצמים. כך נמנעים מיצירה חוזרת של אובייקטים ומנרמול חוזר של אותו וקטור, ולכן מספר החישובים המיותרים קטן וזמן הריצה משתפר, בלי לשנות את התוצאה של האלגוריתם.

נשקלה גם אפשרות להשתמש בספרייה `NumPy` עבור חלק מהחישובים הווקטוריים, מכיוון שהיא יעילה במיוחד בעבודה על מערכים ובביצוע פעולות מספריות בצורה וקטורית. עם זאת, שימוש בה היה דורש שינוי רחב יותר במבנה הקוד. מאחר שהצלחנו להשיג שיפור משמעותי באמצעות שינויים מקומיים ופשטים יחסית, בחרנו לשמור על המבנה המקורי של ה-benchmark ולא להוסיף ספרייה חיצונית נוספת.

השוואת ביצועים

לאחר ביצוע שתי האופטימיזציות הרצנו שוב את אותו `profiling` והשווינו את התוצאות לגרסה המקורית. זמן הריצה של 30 הרצות של `render` ירד מ-56.661 שניות בגרסה המקורית ל-41.281 שניות בגרסה האופטימלית. כלומר, זמן הריצה הכולל קטן בכ-27.14%.

באופטימיזציה של ה-`__slots__`, השינוי אינו קשור לפונקציה מסוימת אלא לאופן שבו אובייקטים ממספר מחלקות נשמרים ונגישים לשדות שלהם. לכן לא ניתן להצביע ב-

Flame Graph על פונקציה אחת שמייצגת את השיפור הזה. ניתן לראות את ההשפעה שלו כחלק מהירידה בזמן הריצה הכולל של ה-benchmark. עם זאת, מכיוון ששתי האופטימיזציות נמדדו יחד, אי אפשר לייחס את כל הירידה של 27.14% ל-__slots__ בלבד.

את ההשפעה של האופטימיזציה ב-lightIsVisible ניתן לראות בצורה ישירה יותר ב-Flame Graph. הזמן של lightIsVisible, כולל הפונקציות שנקראות מתוכה, ירד מ-18.995 שניות בגרסה המקורית ל-8.599 שניות בגרסה האופטימלית. מדובר בירידה של כ-54.73%.

ניתן לראות את השיפור בצורה ברורה גם בפעולה p-l, שמתבצעת באמצעות Point.__sub. בקוד המקורי פעולה זו הייתה בתוך יצירת ה-Ray בתוך הלולאה של lightIsVisible, ולכן בענף זה היא בוצעה 1,786,386 פעמים, וזמן הריצה שלה יחד עם הפעולות שמתבצעות מתוכה היה כ-2.656 שניות. לאחר הוצאת יצירת הקרן מהלולאה, אותה פעולת חיסור בוצעה רק 224,941 פעמים, וזמנה ירד לכ-0.332 שניות. כלומר, מספר הקריאות קטן בערך פי 7.94, וזמן הריצה של ענף זה קטן בכ-87.5%.

אותה תופעה נראית גם ב-Ray.__init. בגרסה המקורית, כאשר יצירת ה-Ray הייתה בתוך הלולאה של lightIsVisible, ה-constructor נקרא בענף זה 1,786,386 פעמים, והזמן שלו יחד עם הפונקציות שנקראו ממנו היה כ-6.811 שניות. לאחר האופטימיזציה מספר הקריאות ירד ל-224,941, והזמן ירד לכ-0.860 שניות. גם כאן מדובר בכמעט פי 8 פחות קריאות, ובירידה של כ-87.4% בזמן.

הירידה של כמעט פי 8 במספר הקריאות מתאימה למבנה הסצנה, שמכילה שמונה עצמים - שבע ספרות ו-Halfspace. בקוד המקורי נוצר Ray חדש בכל איטרציה של הלולאה, ואילו לאחר האופטימיזציה הוא נוצר פעם אחת בלבד לכל קריאה ל-lightIsVisible. היחס אינו בדיוק 8 מכיוון שבמקרים מסוימים הלולאה מסתיימת מוקדם כאשר נמצא עצם שחוסם את מקור האור.

לכן ה-Flame Graph לאחר האופטימיזציה מראה בצורה ברורה שהוצאת יצירת ה-Ray מהלולאה ביטלה עבודה חוזרת: גם פעולת החיסור p-l וגם יצירת ונרמול ה-Ray מתבצעות כמעט פי 8 פחות פעמים. יחד עם השיפור במבנה האובייקטים באמצעות __slots__, זמן הריצה הכולל של ה-benchmark ירד בכ-27.14%, זהו השיפור הכולל של שתי האופטימיזציות יחד.

הצעת המאיץ החומרתי

בחרנו לממש מאיץ חומרתי עבור חישובי החיתוך בין קרניים לספרות, המתבצעים בפונקציה intersectionTime

הפונקציה מקבלת קרן וספרה, בודקת האם הקרן חותכת את הספרה, ובמקרה שקיים חיתוך, מחשבת את הפרמטר t המייצג את מיקום נקודת החיתוך לאורך הקרן. במימוש המקורי ב-python, עבור כל קרן מתבצעת קריאה לintersectionTime עבור כל אחת מהספרות בסצנה. במימוש החומרתי המוצע, נתוני שבע הספרות נשמרים מראש במאיץ החומרתי. עבור כל קרן, המאיץ מבצע את חישובי החיתוך מול הספרות באמצעות pipeline.

לאחר חישוב החיתוכים המאיץ בוחר את החיתוך הואלידי הקרוב ביותר ומחזיר לתוכנה את הפרמטר t המינימלי ואת האינדקס של הספירה המתאימה. שאר החישובים של raytracing נשארים בתוכנה והחומרה מיועדת להאצת חישוב החיתוך של הקרניים מול הספרות בלבד.

מיפוי כניסות ויציאות

כניסות:

- clk – שעון המערכת
- rst_n – אות איפוס המשמש לאתחול מכונת המצבים. (פעיל במצב נמוך)
- $setup_we$ – אות המאפשר כתיבה לצורך טעינת פרמטרי הספירות למודול האחסון.
- $setup_addr$ – הכתובת אליה רוצים לכתוב, מחושבת לפי המיקום בתוך הנתונים של הספירה.
- $setup_wdata$ – המידע שנכתב לכתובת הזיכרון שנבחרה.
- ray_start – אות שגורם למאיץ להתחיל לעבוד ולדגום את פרמטרי הקרן.
- $ray_point_x/y/z$ – נקודת המוצא של הקרן הנכנסת.
- $ray_dir_x/y/z$ – רכיבי וקטור הכיוון של הקרן.

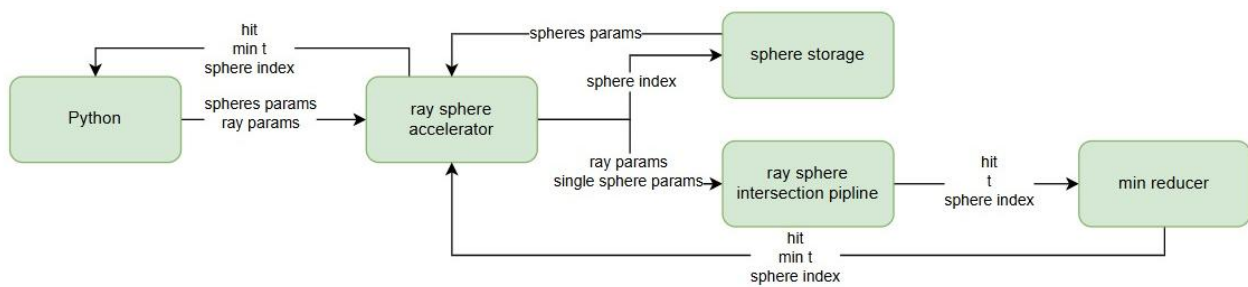
יציאות:

- $ready$ – עולה ל-1 כאשר המאיץ מוכן לקבל את ray_start חדש.
- $result_valid$ – נדלק כאשר תהליך מציאת המינימום מסתיים עבור כל 7 הספירות ונשאר דלוק עד לקבלת הקרן הבאה.
- $result_sphere_index$ – האינדקס של הספירה הקרובה ביותר שיש לה חיתוך עם הקרן.
- $result_t$ – מרחק החיתוך לספירה הקרובה ביותר.

סוגי נתונים:

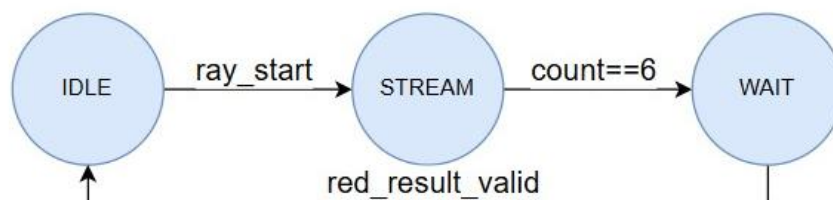
- אותות בקרה וסטטוס הבנויים מביט 1 – $clk, rst_n, setup_we, ray_start, ready,$ $result_valid$
- כתובות ברוחב 5 ביטים – $setup_addr$
- כתובות ברוחב 3 ביטים – $result_sphere_index$
- נתונים מסוג $real$ בגודל 64 ביטים – $ray_point_x/y/z, ray_dir_x/y/z,$ $setup_wdata, result_t$

ארכיטקטורת החומרה



כפי שניתן לראות בשרטוט החומרה מורכבת מ4 חלקים:

- Sphere storage – מודול ששומר את נתוני 7 הספירות. עבור כל ספירה נשמרים מרכז הספירה והרדיוס בריבוע. כאשר המודול מקבל אינדקס של ספירה ומוציא את הנתונים שלה.
- Ray sphere intersection pipeline – זהו datapath העיקרי של התכנית והוא מורכב מ5 שלבים: חישוב הפרש בין מרכז הספירה לנקודת הקרן, ביצוע פעולות הכפל, חישוב המכפלות הסקלריות, חישוב הדסקרימיננטה ולבסוף חישוב השורש. מודול זה ממומש בpipeline מה שמאפשר לספירות שונות להיות בשלבי חישוב שונים באותו זמן וכך ליצור מקביליות.
- Min reducer – מודול שמטרתו למצוא את הספירה שהקרובה ביותר על ידי מציאת נקודת החיתוך המינימלית. מודול זה מחזיר את האינדקס של הספירה עם המרחק המינימלי ואת המרחק מהקרן.
- Ray sphere accelerator – זהו המודול שמממש את מכונת המצבים ולה 3 מצבים: IDLE בו החומרה ממתינה לקרן חדשה, STREAM בו הספירות מוזנות בזז אחר זז לpipeline ו-WAIT שבו ממתנים לסיום החישובים ולקבלת התוצאה מהreducer. כאשר התוצאה מוכנה היא מועברת למוצא והחומרה חוזרת ל-IDLE.



ממשק בין החומרה לתוכנה

במימוש המקורי כל החישובים מתבצעים בפייתון. לאחר הוספת המאיץ החומרתי, התוכנה נשארת אחראית על תהליך raytracen, אך החישובים הקשורים למציאת החיתוך הקרוב ביותר של הקרן עם הספרות מועברים לחומרה. בפועל, החומרה מחליפה לא רק את הקריאות לintersection_time עבור הספרות, אלא גם את חלק מהלוגיקה של ray_colour שבוחרת את החיתוך הקרוב ביותר מביניהן. לפני תחילת עיבוד הקרניים, התוכנה תטען לחומרה את נתוני שבע הספרות בסצנה. עבור

כל ספרה ייטענו מיקום נקודת המרכז וערך הרדיוס בריבוע, והנתונים יישמרו בזיכרון הפנימי של החומרה. מכיוון שנתוני הספרות אינם משתנים במהלך יצירת התמונה, ניתן לטעון אותם פעם אחת ולהשתמש בהם עבור כל הקרניים.

עבור כל קרן, התוכנה שולחת לחומרה את נקודת ההתחלה ואת וקטור הכיוון שלה ומפעילה את החישוב. החומרה מבצעת את בדיקות החיתוך מול שבע הספרות ומחזירה האם נמצא חיתוך, את אינדקס הספרה הקרובה ביותר ואת המיקום של החיתוך.

החישוב מול הרצפה (halfspace) נשאר בתוכנה. לאחר קבלת התוצאה מהחומרה, התוכנה מחשבת את החיתוך עם הרצפה ומשווה אותו לערך ה־z שהתקבל מהחומרה ובהתאם למינימום המתקבל raycolour ממשיכה את שאר החישובים.

את התקשורת בין החומרה לתוכנה ניתן לממש באמצעות i/o memory-mapped. בשיטה זו מוגדרים רגיסטרים של החומרה שממופים לכתובות בזיכרון של המעבד. התוכנה יכולה לכתוב לכתובות אלו את נתוני הספרות והקרן, להפעיל את החומרה ולבדוק באמצעות אותו בקרה כמו valid ready handshake האם החומרה מוכנה לקבל קרן חדשה והאם תוצאת החישוב מוכנה. לאחר סיום החישוב, התוכנית קוראת דרך אותו ממשק את תוצאת החיתוך.

במימוש הנוכחי ממשק mmio לא ממומש, אלא החומרה מספקת את אותות הקלט, הפלט והבקרה הנדרשים לצורך חיבור כזה.

הצדקת השיפור החומרתי

מקוד Python והפרופיל שביצענו ניתן לראות שיש פונקציה שמתבצעת פעמים רבות והיא intersectionTime. פונקציה זו מחשבת עבור כל קרן וכל עצם את המרחק בניהם. מכיוון שפעולה זו מתבצעת על כל קרן בחיתוך עם כל עצם היא מתבצעת המון פעמים ולכן ליצור מאיץ לחלק זה יתרום לשיפור הקוד. המאיץ שבנינו משפר את היעילות בכמה אספקטים:

1. עבור כל קרן, החישובים מול הספרות מתבצעים ב־pipeline, כך שמספר חישובי חיתוך יכולים להימצא בשלבים שונים של החישוב במקביל, ובכך מתקבל throughput גבוה יותר.
2. טוענים את נתוני הספרות פעם אחת למאיץ וכך מצמצמים את התקורה במעבר מהחומרה לתוכנה ומשפר את הביצועים.
3. מכיוון שתוצאות הקרניים לא משפיעות אחת על השניה ניתן לייצר מספר מאיצים חומרתיים וכך להריץ מספר קרניים במקביל ולשפר את זמני הריצה.

החיסרון בפתרון שהצענו הוא שאנחנו עדיין תלויים במימוש חומרתי של פונקציית sqrt והביצועים הסופיים יהיו תלויים במימוש זה.

מה־profiling התקבל כי Sphere.intersectionTime, יחד עם הפונקציות שנקראות מתוכה, מהווה כ־40.4% מזמן הריצה הכולל של ה־benchmark. לכן, לפי חוק אמדל, גם במקרה אידאלי שבו זמן החישוב של חלק זה היה מתאפס לחלוטין, השיפור המקסימלי האפשרי בזמן הריצה הכולל הוא בערך פי 1.68.

החישוב לפי חוק אמדל הוא:

$$P = 22.87 / 56.66 \approx 0.404$$

$$S_{total} = 1 / ((1 - P) + P / S_{HW})$$

$$S_{\max} = 1 / (1 - 0.404) = 1 / 0.596 \approx 1.68$$

כאשר P הוא החלק היחסי של זמן הריצה שאותו מאיצים. במקרה האידאלי מניחים שההאצה של החלק החומרתי שואפת לאינסוף, ולכן האיבר P / S_{HW} שואף לאפס. מכאן מתקבל שהשיפור הכולל המקסימלי האפשרי הוא בערך פי 1.68.

במאיץ החומרתי המוצע חישוב החיתוך בין קרן לספרה מחולק למספר שלבים בצורת pipeline. לאחר מילוי ה-pipeline ניתן להזין חישוב חדש בכל מחזור שעון, ולכן ניתן לעבד את שבע הספרות ברצף בקצב גבוה יותר מאשר ביצוע שבע קריאות נפרדות לפונקציה ב-Python. אם נסמן את זמן ההשהיה של ה-pipeline ב- L מחזורי שעון, אז עיבוד של שבע ספרות יסתיים בקירוב לאחר $L+6$ מחזורים.

הערכה זו מניחה שכל שלב ב-pipeline תוכנן כך שיוכל להשלים את פעולתו בתוך מחזור שעון אחד בתדר היעד. בפועל, פעולות מורכבות כמו floating point ושורש ריבועי עשויות לדרוש מספר מחזורים או pipeline פנימי, ולכן ה-latency הסופי תלוי במימוש החומרתי בפועל. בנוסף, השיפור הכולל יושפע גם מתקורות התקשורת בין התוכנה לחומרה.

לכן ניתן לקבוע שההאצה של חישובי החיתוך עם הספרות יכולה לצמצם משמעותית את הזמן של החלק הזה, אך השיפור הכולל של ה-benchmark חסום לכ-1.68 לפי חוק אמדל, והערך בפועל יהיה תלוי ב-latency וב-throughput של המאיץ ובתקורות התקשורת.

טרייד אוף של ביצועים, שטח והספק:

השימוש במאיץ חומרתי וב-pipeline מאפשר לשפר את הביצועים על ידי ביצוע מספר שלבי חישוב במקביל והגדלת ה-throughput. עם זאת, שיפור זה דורש יותר משאבי חומרה, מכיוון שיש צורך ברגיסטרים בין שלבי ה-pipeline וביחידות חישוב עבור פעולות כמו כפל, חיבור ושורש ריבועי. כתוצאה מכך, הגדלת המקביליות יכולה להגדיל את שטח החומרה (area) ואת צריכת ההספק (power). מצד שני, שימוש ב-pipeline יחיד עבור שבעת הכדורים, במקום מימוש יחיד חישוב נפרדת לכל ספרה, מצמצם את כמות משאבי החומרה ואת צריכת ההספק. החיסרון הוא שהכדורים מוזנים ל-pipeline אחד אחרי השני ולא כולם בו-זמנית, ולכן נדרש זמן רב יותר לעיבוד כל הכדורים עבור קרן מסוימת.

בנוסף, הגדלת תדר העבודה יכולה לשפר את הביצועים, אך תדר גבוה יותר עלול להגדיל את צריכת ההספק ולהקשות על עמידה בדרישות התזמון. לכן, הפתרון שלנו מהווה פשרה בין ביצועים לבין מורכבות החומרה וצריכת ההספק.

מבין הפעולות במאיץ, חישוב השורש הריבועי (sqrt) הוא הפעולה המורכבת ביותר, ולכן בתכנון נבחר את תדר העבודה בהתאם ליכולת של יחידת ה-sqrt למעשה בהתאם לשלב האיטי ביותר ב-pipeline. יחידת ה-sqrt אינה יכולה להשלים את החישוב במחזור אחד, ניתן לממש גם אותה כ-pipeline פנימי או כמספר שלבים.

ניתוח הביצועים של ה-Raytrace הראה שחלק משמעותי מזמן הריצה נובע מפעולות שחוזרות מספר רב של פעמים, ובפרט מחישובי חיתוך בין קרניים לספרות ומחישובי תאורה. באמצעות שתי אופטימיזציות תוכנתיות יחסית פשוטות - שימוש ב-slots במחלקות מרכזיות והוצאת יצירת ה-Ray מחוץ ללולאה בפונקציה lightsVisible - זמן הריצה של 30 הרצות של render ירד מ-56.661 שניות ל-41.281 שניות, כלומר שיפור של כ-27.14%, ללא שינוי באלגוריתם המרכזי של ה-benchmark ושמירה כמובן על נכונות הקוד.

בנוסף, מאחר ש-Sphere.intersectionTime נשאר אחד החלקים המשמעותיים ביותר בזמן הריצה ונקרא מיליוני פעמים, הצענו מאיץ חומרתי לחישוב חיתוך בין קרן לספרות ולבחירת החיתוך התקף הקרוב ביותר. מטרת המאיץ היא לבצע את החישובים הרבים בצורה יעילה ומקבילית יותר, בעוד שהתוכנה ממשיכה לנהל את שאר תהליך ה-Ray Tracing. שילוב בין האופטימיזציות התוכנתיות לבין האצת החומרה צפוי להפחית עוד יותר את זמן הריצה, כאשר השיפור בפועל תלוי גם בזמן התקשורת בין התוכנה לחומרה ובמימוש החומרתי של פעולות שונות כגון ה-floating point ושורש ריבועי.

Pyflate

סקירה כללית

pyflate benchmark מבצע פעולה של פריסת נתונים של קובץ דחוס בפורמט bzip2 ומודד את הביצועים של פריסה זו. פענוח הקובץ מתבצע תחילה על ידי קריאה של הביטים ואז ביצוע של פענוח הופמן. פענוח הופמן הוא אלגוריתם דחיסה המבוסס על קידוד באורך שונה כך שהרעיון המרכזי הוא שתווים נפוצים יותר יקבלו קידוד קצר יותר וכך גודל הקובץ קטן. פסענוח הופמן המבוצע בקוד זה כולל בניית טבלאות המרה של תווים שלפיהן מפוענח הקובץ. לבסוף הקוד גם בודק את תקינות הקובץ המתקבל לראות שהפענוח עבר כראוי.

הקוד משתמש בסיפריות הבאות:

- os – לעבודה עם הקובץ אותו רוצים לפענח.
- hashlib – לחישוב מחרוזת MD5 לצורך בדיקת תקינות המידע.
- Struct – להמרת מספרים שונים לאובייקטים של bytes.
- Pyperf – סיפרייה חיצונית שמשמשת למדידת הביצועים

מבני נתונים:

- BitfieldBase, Bitfield, Rbitfield – מחלקות לקריאת הביטים מתוך הקובץ אותו רוצים לפענח.
- HuffmanLength, HuffmanTable, OrderedHuffmanTable – מחלקות למימוש פענוח הופמן. מכילות בתוכן רשימות פנימיות שמאחסנות אובייקטים עם מידע על כל סמל בקובץ (כמו הקוד שלו והאורך של הקוד).
- Lists – מבנה נתונים דינמיים שמשמשים כחופץ לקריאת נתונים מתוך הקובץ ואחסון זמני של המידע.

אנליזה התחלתית

מניתוח flamegraph עולה כי הרוב המוחלט של זמן הריצה מושקע בפונקציה decode_huffman_block. ניתן ללמוד מזה שזאת הפונקציה המרכזית ממנה מתבצעות הקריאות האחרות, ומהסתכלות על הקוד ניתן לראות שזה באמת המצב ומפונקציה זו מנתחים בלוק אחר בלוק בקוד.

נסתכל כעת על שרשרת הקריאות הארוכה ביותר. בצדו השמאלי של הגרף ניתן לראות מגדל של פונקציות הקוראות אחת לשניה, מהסתכלות בקוד רוב הפונקציות האלה מבצעות פעולות יחסית פשוטות, לכן ניתן ללמוד זה שפונקציות אלה נקראות עמים רבות וקוראות אחת לשניה פעמים רבות מה שאומר שזמן רב מתבזבז על עצם המעבר בין פונקציות שונות והקפיצות בניהן.

בנוסף נסתכל על פונקציות רחבות ושטוחות שאין מעליהן מגדלים משמעותיים כמו bwt_reverse ו-move_to_front. מצורה זו של הגרף ניתן ללמוד שפונקציות אלה מבצעות פעולות מורכבות יותר שלוקחות זמן שמושקע בפונקציה עצמה ולא בקריאות לפונקציות אחרות.

אופטימיזציות

אופטימיזציה ראשונה היא מחיקת הפונקציה `_mask` והכנסת התוכן שלה במקומות המתאימים. פונקציה זו היא פונקציה שמבצעת פעולה יחסית פשוטה אך נקראת המון פעמים, לכן אנחנו רוצים לחסוך את `overhead` שיש בקריאה לפונקציה חדשה. דבר דומה מתבצע גם ב-`Class Rbitfield` מימוש פונקציית `needbits` נמנעת מקריאה לפונקציה `_more` ומבצעת בעצמה את הפעולות המבוצעות בפונקציה. אם נסתכל על האנליזה שביצענו קודם אופטימיזציה זו נועדה לשפר את ה"מגדלים" ולהקטין את שרשרת הקריאות לפונקציות כדי לצמצם את התקורה שבכך.

אופטימיזציה נוספת היא שימוש במשתנים לוקאליים אליהם הגישה קלה יותר מאשר שדות של אובייקט. ניתן לראות זאת במימוש של פונקציית `needbits` בתוך `Rbitfield`. במקום לעבוד בתוך הלולאה עם השדות של האובייקט הפונקציה תחילה שומרת את המידע במשתנים פנימיים, מבצעת את הלולאה ואז שומרת את הערכים החדשים בשדות האובייקט חזרה.

אופטימיזציה שלישית היא שימוש בהקצאת זיכרון מראש במקום שימוש ברשימה דינמית שגדלה עם הזמן. כפי שראינו באנליזה הראשונית `bwt_reverse` היא פונקציה שמושקע בה הרבה זמן שמתבטא ומלבן ארוך שאין מעליו כמעט קריאות לפונקציות אחרות. ראינו שיש בפונקציה זו שימוש ברשימה שגדלה בהדרגה באמצעות פעולות `append`. פעולות אלה גוזלות זמן רב (שחלק ממנו ניתן לראות במלבן `append` שמעל פונקציה זו). במקום זה הקצנו מערך בגודל קבוע מראש והערכים מוצבים בו ישירות לפי אינדקס. כך אין צורך בהקצאות זיכרון מחודשות והעתקות מיותרות ויש לנו עבודה נוחה ופשוטה יותר מול הזיכרון.

השוואת ביצועים

ניתן לראות בדוח כי זמן הריצה הממוצע של `benchmark` ירד מ3.16 שניות ל2.80 שניות לאחר אופטימיזציה שזה אומר שהקוד המשופר רץ מהר פי 1.13 דבר שעומד ב7% שיפור שנדרש בפרויקט. בנוסף אם מסתכלים על נתוני הזמנים ניתן לראות כי השתפרנו גם ביציבות הריצה. עבור הקוד המאופטם השגיאה מוערכת ב0.01 שניות כאשר בקוד המקורי היא 0.03. אנחנו מעריכים ששיפור זה נובע מהמעבר לשימוש במערך סטטי מאשר רשימה דינמית שגוזרות פעולות פחות צפויות כמו הגדלת הרשימה.

מהשוואת `flamegraph` של הקוד המאופטם לקוד המקורי נראה בידיוק את התופעות שצפינו. העלמנו לגמרי את הבלוקים של `_mask` ושל `append` כי אנחנו לא משתמשים בפונקציות האלה יותר ובנוסף המעטנו את השימוש ב-`_more`. דבר נוסף שאפשר לשים אליו לב זה שפונקציית `bwt_reverse` תופסת כעת פחות אחוזים מהריצה (מלבן קצר יותר) לאחר ששיפרנו אותה במעבר למערך סטטי.

סיכום

ניתן לראות שבאמצעות כלים להסתכלות על הקוד ועל איזה חלקים בו לוקחים משאבים גדולים, ניתן למפות את צווארי הבקבוק ובכך ליצור שיפורים בקוד שמסתמכים גם על הבנת החומרה וצורת העבודה. באופן כללי השיפורים התמקדו בשני תחומים, הורדת התקורה שבקריאה לפונקציה. יש יתרון בכתיבת פונקציות מרובות לצורך קריאות של הקוד אך לפעמים פעולה זו פוגעת בנו כי כאשר אנחנו קוראים לפונקציה מתבצעות פעולות נוספות

מלבד פעולת הפונקציה עצמה שיוצרות תקורה לא רצויה שיכולה להיות משמעותית ביחס לפעולת הפונקציה. בנוסף ראינו כי שימוש במשתנים מקומיים הנמצאים במערך רציף וקבוע מראש עדיפות על גישה לשדות של אובייקטים שדורשת שליפת האובייקט מהזיכרון וביצוע פעולות עליו להגעה לערך עצמו.

בנוסף התייחסנו במהלך האופטימיזציה לעבודה מול הזיכרון בבחירה במערך סטטי על פני רשימה דינאמית. שכן מערך סטטי גם מממש את עיקרון spatial locality וגם מונע תקורה שמקורה בהגדלת המערך הדינמי בעת הוספת איברים והעתקת המערך במידת הצורך.